

NT-C-014 | Core Python | Course Knowledge Profile

Identity, purpose, prerequisites, scope and boundaries

Category	Programming
Target learners	Beginners and learners who need Python foundations for software, data or automation.
Prerequisites	Basic computer use and logical thinking.
Approved duration	8-10 weeks / 80-120 guided hours.
Final outcome	Write structured, tested Python programs that work with files, databases and APIs.
Assessment profile	Programming/Platform

Approved tools / platforms

Python, standard library, SQL basics, JSON/HTTP APIs, unittest or pytest, Git.

Knowledge boundaries

Include	Quality rule	Exclude / escalate
The eight approved modules, four sections, named tools, projects and mapped entry-level roles.	Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.	Do not treat copied code or syntax recall as proof of implementation skill.
Current product/platform procedures only when version and date are known.	Use primary documentation and record the source version.	Do not invent current commands, prices, tax rates, policies or certification status.
Evidence-based career guidance.	Cite tests, projects, capstone, viva and verified resume evidence.	No guaranteed job, placement, salary, interview or employer claim.

LLM answer pattern: identify the course and version, answer from approved records, cite record IDs, distinguish verified facts from guidance, and state any missing evidence.

NT-C-014 | Core Python | NT-C-014-S01 Deep Knowledge

Modules 1-2 | objectives, concepts, evidence and remediation

Section competency	Interpreter, variables, types and operators; Conditions, loops and comprehensions
Completion evidence	Basic programs; Logic exercises
Section weight	20% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 1 - Python setup

Objective: Interpreter, variables, types and operators

Observable evidence: Basic programs

Concept	Approved knowledge statement	Application behaviour	Evidence
Interpreter	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when interpreter is appropriate; apply it in a Core Python task; verify the result.	Basic programs
variables	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when variables is appropriate; apply it in a Core Python task; verify the result.	Basic programs
types	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when types is appropriate; apply it in a Core Python task; verify the result.	Basic programs
operators	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when operators is appropriate; apply it in a Core Python task; verify the result.	Basic programs

Module 2 - Control flow

Objective: Conditions, loops and comprehensions

Observable evidence: Logic exercises

Concept	Approved knowledge statement	Application behaviour	Evidence
Conditions	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when conditions is appropriate; apply it in a Core Python task; verify the result.	Logic exercises
loops	A design approach that organises behaviour and data around objects and clear responsibilities. In Core Python, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when loops is appropriate; apply it in a Core Python task; verify the result.	Logic exercises
comprehensions	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when comprehensions is appropriate; apply it in a Core Python task; verify the result.	Logic exercises

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Basic programs; Logic exercises, return to Modules 1-2, complete one guided practice and then use a new equivalent test form.

NT-C-014 | Core Python | NT-C-014-S02 Deep Knowledge

Modules 3-4 | objectives, concepts, evidence and remediation

Section competency	Lists, tuples, dictionaries and sets; Functions, scope, packages and reuse
Completion evidence	Data-processing tasks; Modular program
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 3 - Data structures

Objective: Lists, tuples, dictionaries and sets

Observable evidence: Data-processing tasks

Concept	Approved knowledge statement	Application behaviour	Evidence
Lists	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when lists is appropriate; apply it in a Core Python task; verify the result.	Data-processing tasks
tuples	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when tuples is appropriate; apply it in a Core Python task; verify the result.	Data-processing tasks
dictionaries	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when dictionaries is appropriate; apply it in a Core Python task; verify the result.	Data-processing tasks
sets	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when sets is appropriate; apply it in a Core Python task; verify the result.	Data-processing tasks

Module 4 - Functions and modules

Objective: Functions, scope, packages and reuse

Observable evidence: Modular program

Concept	Approved knowledge statement	Application behaviour	Evidence
Functions	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when functions is appropriate; apply it in a Core Python task; verify the result.	Modular program
scope	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when scope is appropriate; apply it in a Core Python task; verify the result.	Modular program
packages	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when packages is appropriate; apply it in a Core Python task; verify the result.	Modular program
reuse	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when reuse is appropriate; apply it in a Core Python task; verify the result.	Modular program

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Data-processing tasks; Modular program, return to Modules 3-4, complete one guided practice and then use a new equivalent test form.

NT-C-014 | Core Python | NT-C-014-S03 Deep Knowledge

Modules 5-6 | objectives, concepts, evidence and remediation

Section competency	Classes, objects and inheritance; File formats, exceptions and logging
Completion evidence	OOP application; Reliable file utility
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 5 - Object-oriented Python

Objective: Classes, objects and inheritance

Observable evidence: OOP application

Concept	Approved knowledge statement	Application behaviour	Evidence
Classes	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when classes is appropriate; apply it in a Core Python task; verify the result.	OOP application
objects	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when objects is appropriate; apply it in a Core Python task; verify the result.	OOP application
inheritance	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when inheritance is appropriate; apply it in a Core Python task; verify the result.	OOP application

Module 6 - Files and errors

Objective: File formats, exceptions and logging

Observable evidence: Reliable file utility

Concept	Approved knowledge statement	Application behaviour	Evidence
File formats	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when file formats is appropriate; apply it in a Core Python task; verify the result.	Reliable file utility
exceptions	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when exceptions is appropriate; apply it in a Core Python task; verify the result.	Reliable file utility
logging	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when logging is appropriate; apply it in a Core Python task; verify the result.	Reliable file utility

Remediation: If the learner cannot explain the purpose, complete the stated task or provide OOP application; Reliable file utility, return to Modules 5-6, complete one guided practice and then use a new equivalent test form.

NT-C-014 | Core Python | NT-C-014-S04 Deep Knowledge

Modules 7-8 | objectives, concepts, evidence and remediation

Section competency	SQL basics, JSON and HTTP API use; Unit tests, Git and documentation
Completion evidence	API/data exercise; Tested Python project
Section weight	30% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 7 - Data and APIs

Objective: SQL basics, JSON and HTTP API use

Observable evidence: API/data exercise

Concept	Approved knowledge statement	Application behaviour	Evidence
SQL basics	A language for defining, querying and controlling relational data. In Core Python, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when sql basics is appropriate; apply it in a Core Python task; verify the result.	API/data exercise
JSON	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when json is appropriate; apply it in a Core Python task; verify the result.	API/data exercise
HTTP API use	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when http api use is appropriate; apply it in a Core Python task; verify the result.	API/data exercise

Module 8 - Testing and capstone

Objective: Unit tests, Git and documentation

Observable evidence: Tested Python project

Concept	Approved knowledge statement	Application behaviour	Evidence
Unit tests	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when unit tests is appropriate; apply it in a Core Python task; verify the result.	Tested Python project
Git	Distributed version control for tracking and reviewing source changes. In Core Python, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when git is appropriate; apply it in a Core Python task; verify the result.	Tested Python project
documentation	A defined competency within Core Python. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when documentation is appropriate; apply it in a Core Python task; verify the result.	Tested Python project

Remediation: If the learner cannot explain the purpose, complete the stated task or provide API/data exercise; Tested Python project, return to Modules 7-8, complete one guided practice and then use a new equivalent test form.

NT-C-014 | Core Python | Skill Ontology & Dependency Map

Atomic skills, learning order and evidence strength

Skill ID	Competency	Depends on	Evidence	Type
NT-C-014-SK01	Python setup	Prerequisite foundation	Basic programs	Critical
NT-C-014-SK02	Control flow	M01 plus earlier foundations	Logic exercises	Critical
NT-C-014-SK03	Data structures	M02 plus earlier foundations	Data-processing tasks	Supporting
NT-C-014-SK04	Functions and modules	M03 plus earlier foundations	Modular program	Supporting
NT-C-014-SK05	Object-oriented Python	M04 plus earlier foundations	OOP application	Supporting
NT-C-014-SK06	Files and errors	M05 plus earlier foundations	Reliable file utility	Critical
NT-C-014-SK07	Data and APIs	M06 plus earlier foundations	API/data exercise	Supporting
NT-C-014-SK08	Testing and capstone	M07 plus earlier foundations	Tested Python project	Critical

Evidence-strength hierarchy

Strength	Evidence source	Use
High	Trainer-observed practical, viva, working source/project files and reproducible output.	May support verified mastery when rubric threshold is met.
Medium	Scored mock tests, reviewed assignments and documented portfolio artifacts.	Supports mastery with corroboration.
Low	Resume claim, self-report, attendance or video completion without demonstration.	May trigger verification; cannot alone support Apply Now.

CareerTours must not treat attendance, time spent or content opened as skill mastery.

NT-C-014 | Core Python | Assessment Knowledge & Sample Items

Non-live examples for LLM training and evaluation

Sample ID	Type	Question	Approved answer / rubric anchor
NT-C-014-S01-EX01	Evidence/application	What evidence best demonstrates the combined competency of Python setup and Control flow?	The learner should provide Basic programs; Logic exercises and explain how the work applies Interpreter, variables, types and operators; Conditions, loops and comprehensions. Recall alone is insufficient.
NT-C-014-S01-EX02	Scenario/remediation	A learner reports completing Control flow but cannot produce Logic exercises. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-014-S01, request Logic exercises, and allow a new equivalent test only after remediation.
NT-C-014-S02-EX01	Evidence/application	What evidence best demonstrates the combined competency of Data structures and Functions and modules?	The learner should provide Data-processing tasks; Modular program and explain how the work applies Lists, tuples, dictionaries and sets; Functions, scope, packages and reuse. Recall alone is insufficient.
NT-C-014-S02-EX02	Scenario/remediation	A learner reports completing Functions and modules but cannot produce Modular program. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-014-S02, request Modular program, and allow a new equivalent test only after remediation.
NT-C-014-S03-EX01	Evidence/application	What evidence best demonstrates the combined competency of Object-oriented Python and Files and errors?	The learner should provide OOP application; Reliable file utility and explain how the work applies Classes, objects and inheritance; File formats, exceptions and logging. Recall alone is insufficient.
NT-C-014-S03-EX02	Scenario/remediation	A learner reports completing Files and errors but cannot produce Reliable file utility. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-014-S03, request Reliable file utility, and allow a new equivalent test only after remediation.
NT-C-014-S04-EX01	Evidence/application	What evidence best demonstrates the combined competency of Data and APIs and Testing and capstone?	The learner should provide API/data exercise; Tested Python project and explain how the work applies SQL basics, JSON and HTTP API use; Unit tests, Git and documentation. Recall alone is insufficient.
NT-C-014-S04-EX02	Scenario/remediation	A learner reports completing Testing and capstone but cannot produce Tested Python project. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-014-S04, request Tested Python project, and allow a new equivalent test only after remediation.

Live bank rule: Generate at least three times the live quantity for each section using the approved Programming/Platform blueprint. Human-review every scored item before initial production release.

NT-C-014 | Core Python | Labs, Projects & Scoring Rubrics

Practical proof of learning and ownership

Level	Approved project	Skills evidenced	Required deliverables
Mini 1	File organiser	Files, functions and errors	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Mini 2	Expense tracker	Python, SQL and CRUD	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Capstone	Automation utility	APIs, testing and documentation	Brief; working/source file or reproducible environment; output; validation notes; reflection.

Standard project rubric

Dimension	Weight	What earns credit
Problem framing	15%	Clear objective, user/business need, assumptions and success criteria.
Technical/design accuracy	25%	Correct methods, appropriate tools and sound decisions.
Completeness	15%	Required features, assets, data or workflow are present.
Testing and validation	15%	Important normal, edge, quality or failure cases are checked.
Evidence and reproducibility	15%	Source/working files, setup, inputs, outputs and version information are available.
Documentation	5%	Concise structure, instructions and limitations.
Viva / ownership	10%	Learner can explain decisions, demonstrate work and respond to a change request.

Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.

NT-C-014 | Core Python | Career Roles, Evidence & Gap Actions

Explainable top-three occupation matching

Rank	Occupation	Default weighted skills	Minimum evidence	Gap action
1	Junior Python Programmer	Python 34% [critical]; OOP 33% [critical]; Git 33%	Tested repository	If a critical skill is below 60 or tested repository is missing, return Improve First or Needs Evidence with the linked module/project.
2	Automation Support Trainee	Files 34% [critical]; APIs 33% [critical]; scripting 33%	Working utility	If a critical skill is below 60 or working utility is missing, return Improve First or Needs Evidence with the linked module/project.
3	Python Application Trainee	Data 34% [critical]; errors 33% [critical]; modular design 33%	Capstone application	If a critical skill is below 60 or capstone application is missing, return Improve First or Needs Evidence with the linked module/project.

Course readiness	45% section mocks + 20% mini projects + 25% capstone + 10% viva.
Role readiness	45% role-relevant mastery + 25% projects/capstone + 20% aligned mocks + 10% verified resume evidence.
Evidence confidence	Show separately; Apply Now requires at least 70.
Resume extraction	Use only relevant skills, projects, education, experience and portfolio evidence; minimise personal data before external model processing.

A student may be Apply Now for one mapped occupation and Improve First for another. Return independent evidence and gaps for each role.

NT-C-014 | Core Python | FAQs, Misconceptions & LLM Response Pack

Course-specific retrieval and response patterns

Approved FAQ	Answer
Who is Core Python for?	Beginners and learners who need Python foundations for software, data or automation.
What should I know before starting?	Basic computer use and logical thinking.
How long is the approved learning journey?	8-10 weeks / 80-120 guided hours.
What will I be able to demonstrate?	Write structured, tested Python programs that work with files, databases and APIs.
Does completion guarantee a job?	No. Completion and role readiness are evidence-based guidance; employment, placement, salary and employer selection are not guaranteed.
What if I fail a section?	CareerTours identifies the failed skill tags and module, assigns targeted practice, and permits an equivalent retest after remediation.

Misconception	Approved correction
Attendance proves mastery	False - require tests and observable practical/project evidence.
One total score proves every role fit	False - calculate each occupation using its weighted skills and evidence.
The LLM can add missing details	False - retrieve approved records or state that evidence is insufficient.
Course completion guarantees employment	False - never claim guaranteed job, placement, salary or interview.
Core Python knowledge never changes	False - version-sensitive tools, laws, taxes, security and platform procedures require review.

Retrieval metadata

```
course_code=NT-C-014; corpus_version=3.0;
record_types=course_profile,module,section,skill,assessment_sample,project,occupation_mapping,faq,misconception;
approval_status=approved; effective_date=2026-08-12
```